

MSc 2025-2026 · Data Architecture for AI

Labo Santé

Architecture de données containerisée
pour un projet IA

✓ **Session 1 — DONE + VALIDÉ**

Livrable Session 1 — Modélisation, premiers conteneurs & validation locale

Aline Maire · 9 juin 2026

Repo : github.com/Almaire-Lab/labo-sante

Sommaire

1. Cahier des charges
2. Schéma SQL (PostgreSQL) — export dbdiagram.io
3. Schéma NoSQL (MongoDB) — document JSON
4. Diagramme des services & interactions
5. **Validation locale — stack qui tourne**

1. Cahier des charges

1.1 Contexte

Un **laboratoire de santé fictif** souhaite centraliser ses données médicales aujourd'hui éclatées entre plusieurs fichiers et logiciels métier. L'objectif est de bâtir une plateforme :

- **fiable** — intégrité référentielle, pas de perte au redémarrage
- **scalable** — capable d'ingérer plusieurs millions de consultations
- **administrable** — interfaces web pour explorer les données
- **observable** — monitoring temps réel des services
- **portable** — tout est containerisé, déployable en une commande

À terme, ces données alimenteront des modèles IA (prédiction de prescription, détection d'interactions médicamenteuses, NLP sur les consultations).

1.2 Données à gérer

Domaine	Type	Stockage cible	Justification
Médecins	structuré	PostgreSQL	schéma stable, jointures
Patients	structuré	PostgreSQL	schéma stable, intégrité forte
Médicaments	structuré	PostgreSQL	référentiel, peu volatile
Prescriptions	structuré (N-N)	PostgreSQL	relations patient ↔ médecin ↔ médicament
Consultations	semi-structuré	MongoDB	symptômes/diagnostics de longueur et structure variables, pas de jointures nécessaires

1.3 Besoins fonctionnels

- **F1** — Enregistrer/consulter médecins, patients, médicaments via une UI (pgAdmin).
- **F2** — Créer des prescriptions liant patient + médecin + 1..N médicaments.
- **F3** — Stocker les consultations (symptômes libres, diagnostic structuré, notes) avec lien `patient_id` vers PostgreSQL.
- **F4** — Ingérer en masse des données simulées (jusqu'à 6 M de consultations) pour tester la volumétrie.
- **F5** — Re-exécuter le pipeline sans créer de doublons (idempotence).
- **F6** — Consulter en temps réel l'état des services et leurs logs (Portainer).

1.4 Besoins techniques

Catégorie	Exigence
Conteneurisation	<code>docker compose up -d</code> démarre tous les services
Persistance	volumes Docker pour Postgres et Mongo (survie à <code>down</code>)
Réseau	un réseau Docker isolé, communication inter-services par nom de service
Secrets	aucun mot de passe en dur — variables via fichier <code>.env</code> (jamais commité)
Ordonnancement	<code>depends_on</code> + <code>healthcheck</code>
Observabilité	pgAdmin, Mongo Express, Portainer, <code>docker stats</code>
Reproductibilité	<code>.env.example</code> , <code>Dockerfile</code> versionnés ; <code>.gitignore</code> strict

1.5 Critères d'acceptation

-  `docker compose up -d` démarre tous les services sans erreur.
-  pgAdmin sur `localhost:5050` , Mongo Express sur `:8081` , Portainer sur `:9000` .
-  Schéma SQL créé automatiquement au 1er démarrage.
-  Ingestion 60k / 600k / 6M lignes mesurée (*Session 2*).
-  Aucune perte après `down && up` (volumes — testé en *Session 2*).
-  Aucun secret en clair dans le dépôt Git.

2. Schéma SQL (PostgreSQL)

Source dbdiagram.io éditable : <https://dbdiagram.io/d/Labo-Sante-Schema-SQL-6a27e6f88eb8ca4bfe863a2b>

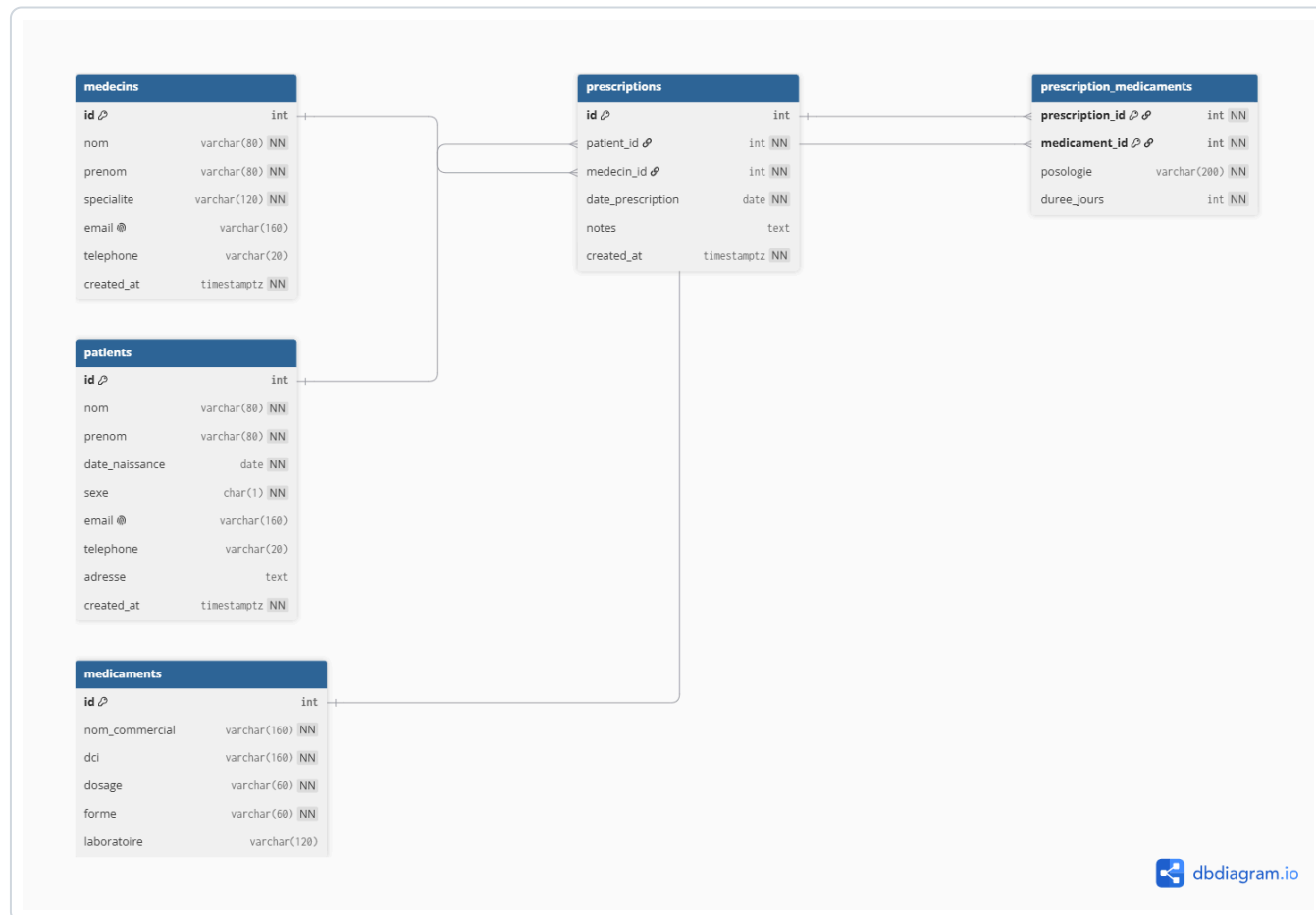


Schéma SQL — 5 tables, FK explicites, table de liaison N-N.

2.1 Tables

Table	Rôle	Clés
medecins	référentiel praticiens	PK id , UNIQUE email
patients	référentiel patients	PK id , UNIQUE email , CHECK sexe IN ('M', 'F', 'X')
medicaments	référentiel médicaments	PK id , UNIQUE (nom_commercial, dosage, forme)
prescriptions	actes de prescription	PK id , FK patient_id , FK medecin_id
prescription_medicaments	liaison N-N	PK composite (prescription_id, medicament_id)

2.2 Contraintes & index

- **FK** avec `ON DELETE RESTRICT` (cascade sur la liaison N-N).
- **CHECK**: `duree_jours > 0`, `sexe IN ('M','F','X')`.
- **Index**: `patient_id`, `medecin_id`, `date_prescription`, `dci`.
- **Auto-exécution**: `schema.sql` monté dans `/docker-entrypoint-initdb.d/` → appliqué au 1er `docker compose up`.

3. Schéma NoSQL (MongoDB)

Collection unique `consultations` . Exemple de document :

```
{
  "_id": "ObjectId('6a27f6d0a7c10f321e9df8a3')",
  "patient_id": 1428,
  "medecin_id": 12,
  "date": "2026-06-09T11:19:44Z",
  "symptoms": [
    { "label": "toux sèche",          "duree_jours": 10, "intensite": "moderee" },
    { "label": "fièvre",             "temperature_c": 38.4 }
  ],
  "diagnosis": {
    "primary": { "code_cim10": "J20.9", "label": "Bronchite aiguë" },
    "secondary": [{ "code_cim10": "R05", "label": "Toux" }],
    "severity": "moderee"
  },
  "notes": "Auscultation : râles bronchiques bilatéraux."
}
```

3.1 Champs obligatoires

Champ	Type	Rôle
<code>patient_id</code>	int	lien vers <code>patients.id</code> (PostgreSQL) — jointure applicative
<code>medecin_id</code>	int	lien vers <code>medecins.id</code> (PostgreSQL)
<code>date</code>	ISODate	date de la consultation
<code>symptoms</code>	array<object>	longueur variable
<code>diagnosis</code>	object	structure variable

3.2 Index recommandés (créés en local)

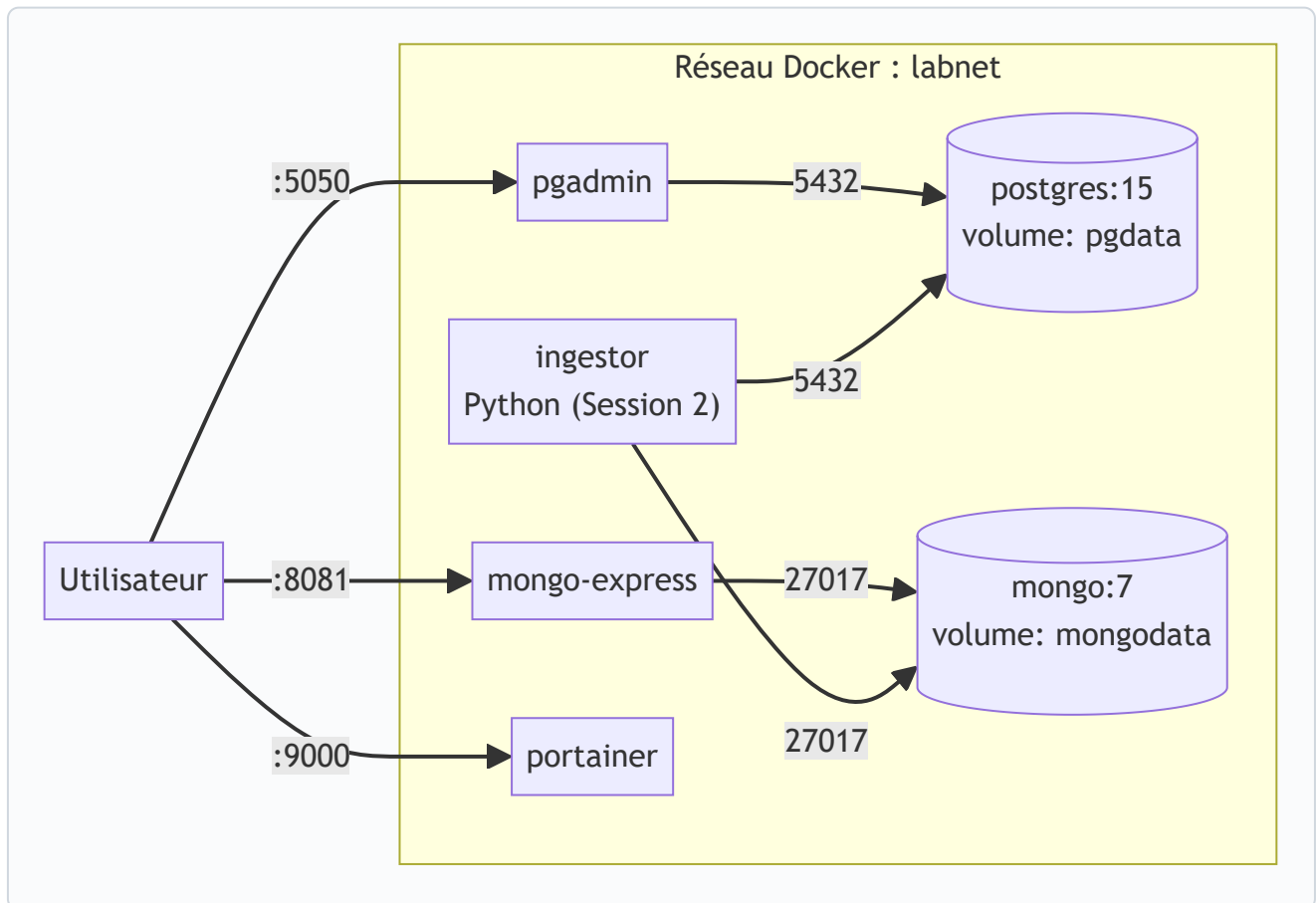
- `{ patient_id: 1 }`
- `{ date: -1 }`
- `{ "diagnosis.primary.code_cim10": 1 }`

3.3 Justification NoSQL

- `symptoms` et `diagnosis` ont une **structure variable**.
- **Pas de jointure complexe** nécessaire sur ce document.
- `patient_id` assure le **lien logique** avec PostgreSQL.
- Évolution facile du modèle **sans migration**.

4. Diagramme des services & interactions

Tous les services tournent dans un **réseau Docker isolé** (`labnet`). Ils communiquent par leur **nom de service** (pas `localhost`). Les volumes nommés assurent la persistance.



4.1 Inventaire des services

Service	Image	Port hôte	Volume	Dépend de
postgres	postgres:15-alpine	127.0.0.1:5432	pgdata	—
mongo	mongo:7	127.0.0.1:27017	mongodata	—
pgadmin	dpage/pgadmin4	127.0.0.1:5050	pgadmin	postgres (healthy)
mongo-express	mongo-express:1	127.0.0.1:8081	—	mongo (healthy)
portainer	portainer/portainer-ce	127.0.0.1:9000	portainer	—

4.2 Healthchecks

- postgres** : `pg_isready -U $POSTGRES_USER -d $POSTGRES_DB`
- mongo** : `mongosh --eval "db.adminCommand('ping')"`

5. Validation locale — la stack tourne

La stack a été démarrée localement avec `docker compose up -d` et tous les critères de Session 1 sont vérifiés.

5.1 `docker compose ps` — 5 services UP

NAME	IMAGE	STATUS
labo-postgres	postgres:15-alpine	Up (healthy)
labo-mongo	mongo:7	Up (healthy)
labo-pgadmin	dpage/pgadmin4:latest	Up
labo-mongo-express	mongo-express:1	Up
labo-portainer	portainer/portainer-ce:latest	Up

5.2 Schéma SQL auto-crée — 5 tables

```
$ docker exec labo-postgres psql -U lab_admin -d labo -c "dt"
```

Schema	Name	Type	Owner
public	medecins	table	lab_admin
public	medicaments	table	lab_admin
public	patients	table	lab_admin
public	prescription_medicaments	table	lab_admin
public	prescriptions	table	lab_admin

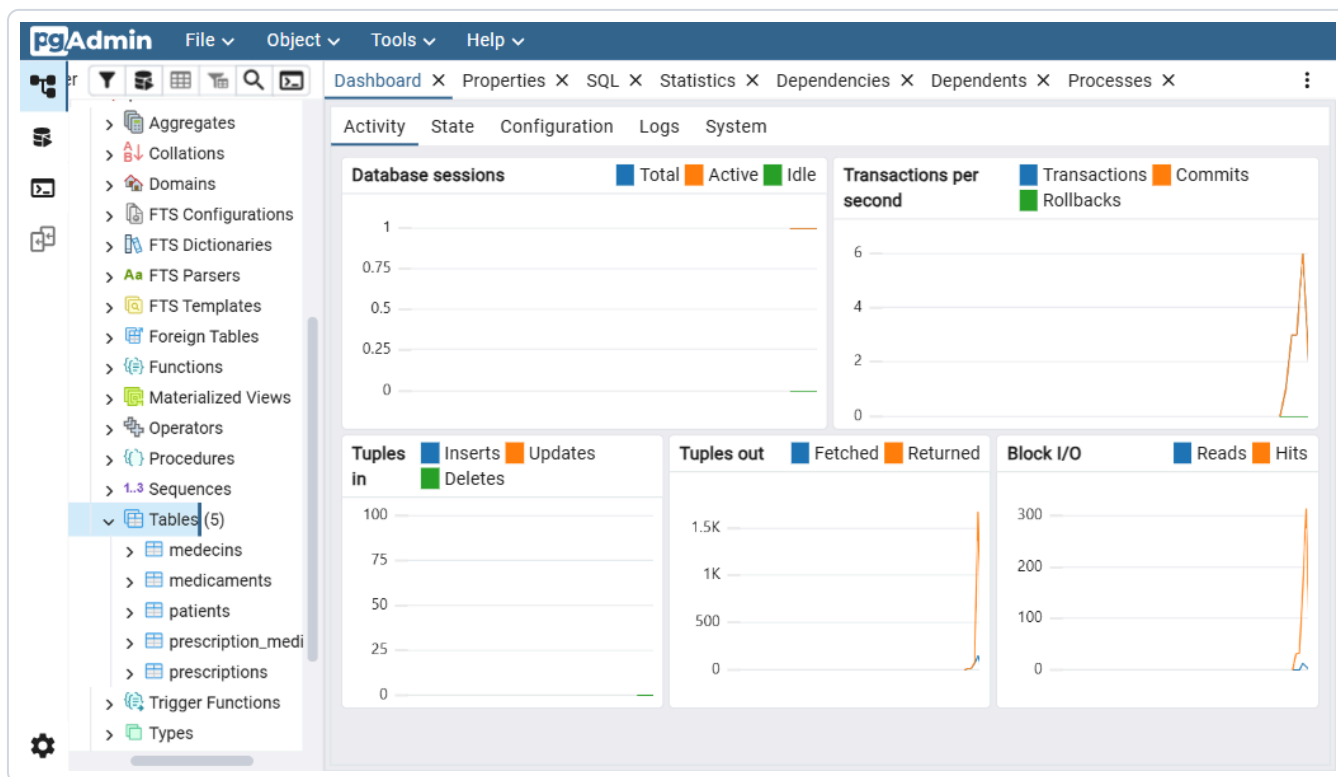
(5 rows)

5.3 MongoDB — collection `consultations` créée + index

```
$ docker exec labo-mongo mongosh --quiet --eval '
  db.getSiblingDB("labo").consultations.countDocuments()'
1                                     # 1 document seed

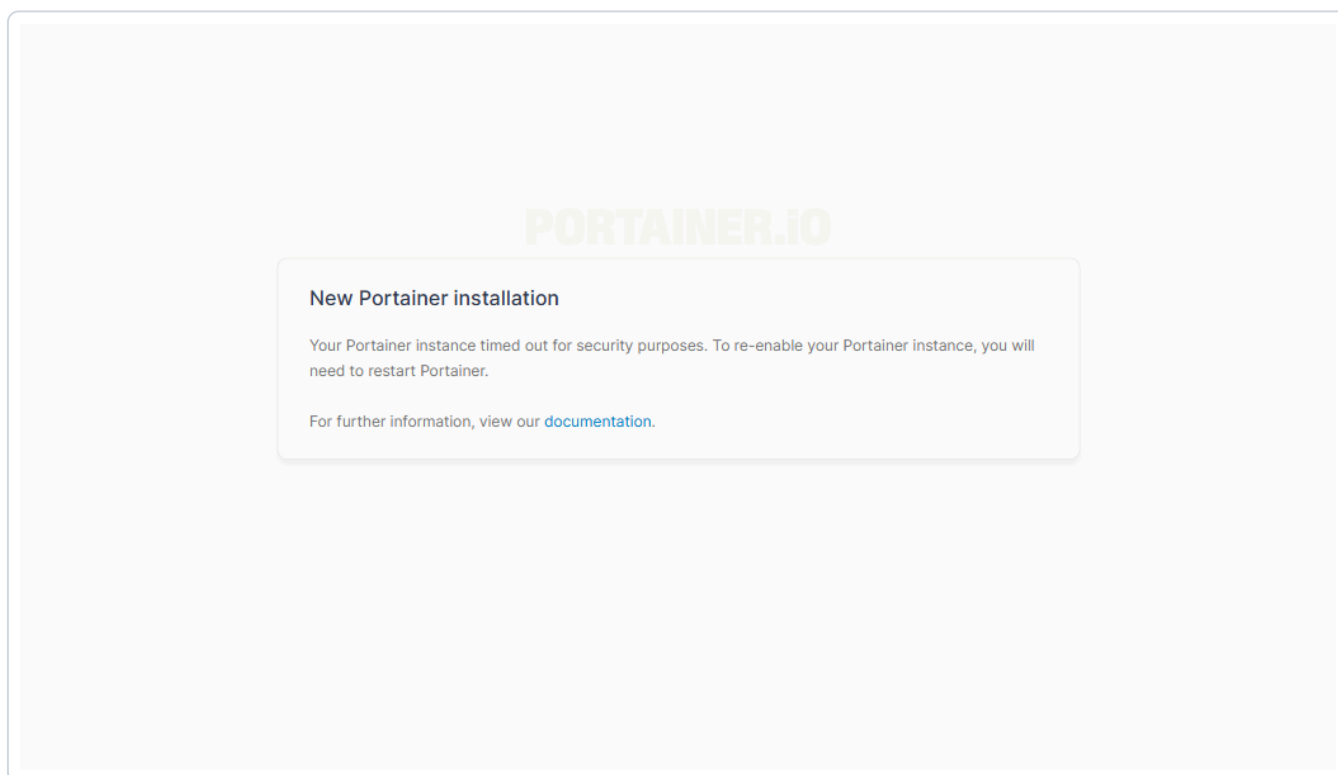
$ db.consultations.getIndexes()
[ "_id_", "patient_id_1", "date_-1" ]      # 3 index créés
```

5.4 pgAdmin — serveur `labo` connecté, 5 tables visibles



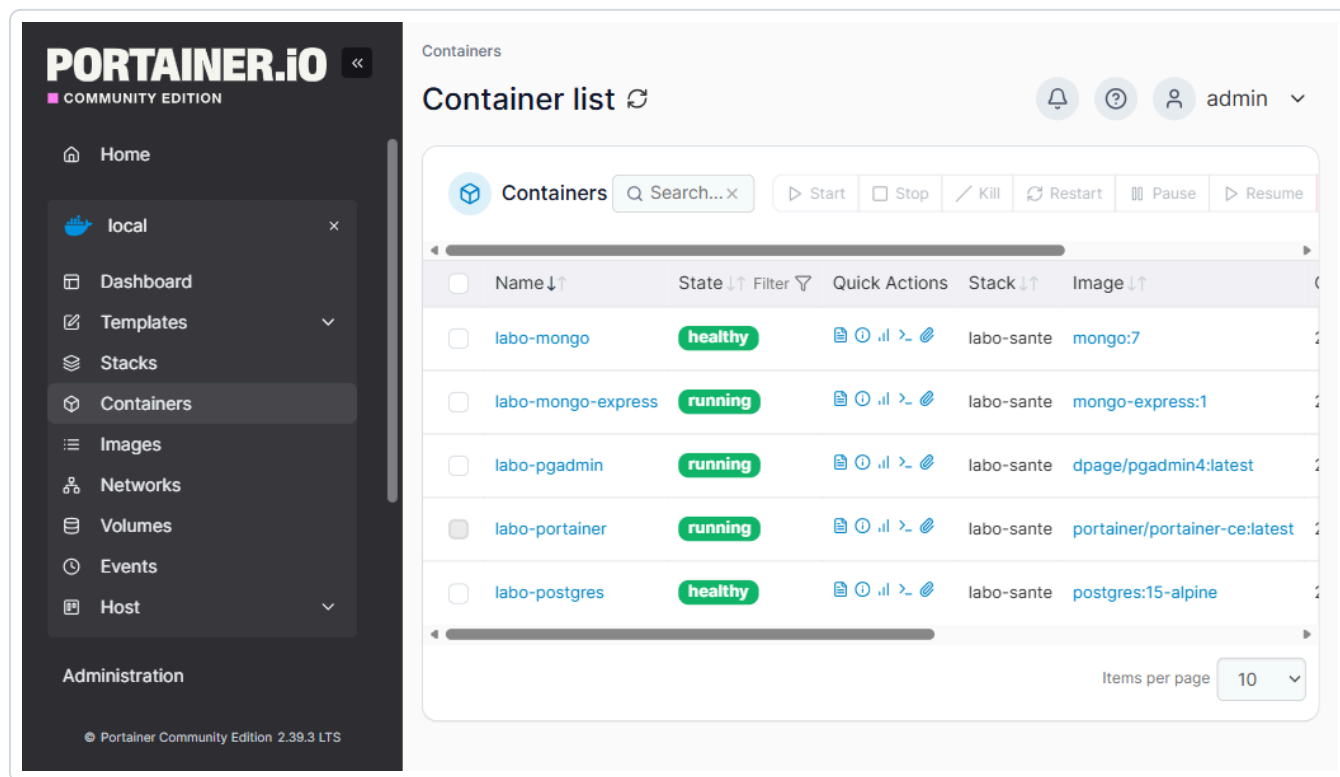
Tables (5) déployées dans le schéma `public` · Dashboard activité du serveur.

5.5 Mongo Express — DB `labo` > collection `consultations`



1 document inséré · 3 index actifs (`_id` , `patient_id` , `date`).

5.6 Portainer — 5 conteneurs running, 2 healthy



Stack `labo-sante` · 5 containers · 5 volumes · 5 images (2.9 GB).

5.7 `docker stats` — empreinte au repos

NAME	CPU %	MEM USAGE / LIMIT	NET I/O	BLOCK I/O
labo-pgadmin	0.11%	261.9 MiB / 7.67 GiB	238 kB / 17.4 MB	0 B / 19.9 MB
labo-mongo-express	0.00%	44.2 MiB / 7.67 GiB	331 kB / 768 kB	0 B / 0 B
labo-postgres	0.00%	37.1 MiB / 7.67 GiB	122 kB / 26.6 kB	0 B / 54.4 MB
labo-portainer	0.00%	22.1 MiB / 7.67 GiB	302 kB / 356 kB	0 B / 0 B
labo-mongo	1.01%	181 MiB / 7.67 GiB	26.4 kB / 261 kB	0 B / 3.1 MB
TOTAL	~1.1 %	~ 546 MiB		~ 78 MB BLOCK

5.8 Critères de Session 1 — tableau de conformité

Critère	Statut	Preuve
<code>docker compose up -d</code> démarre sans erreur	✓	§ 5.1
5 services UP, postgres + mongo healthy	✓	§ 5.1
5 tables Postgres auto-crées via <code>init.sql</code>	✓	§ 5.2
pgAdmin accessible & connecté au serveur <code>labo</code>	✓	§ 5.4
Mongo Express accessible · DB <code>labo</code> · collection visible	✓	§ 5.5

Critère	Statut	Preuve
Portainer accessible & dashboard fonctionnel	✓	§ 5.6
Aucun mot de passe en dur (variables <code>.env</code>)	✓	<code>.env.example</code> + <code>.gitignore</code>
Reproductibilité : <code>cp .env.example .env && docker compose up -d</code>	✓	README §«Démarrer le projet»

Conclusion Session 1 : tous les livrables sont produits, versionnés sur GitHub, et l'infrastructure est opérationnelle. La Session 2 pourra démarrer immédiatement sur la base de cet environnement validé en branchant le service `ingestor` (Python · `psycpg2` · `pymongo`) sur les bases déjà *healthy*.